

7th International Conference on AI in Computational Linguistics

LLM-based 3D Model Generation of MHE for OpenSCAD

Nouran ELhadad^a, Amal Aboulhassan^b, Yomna M.I. Hassan^{*a}^aGerman University in Cairo, Cairo, Egypt^bGerman International University, Cairo, Egypt

Abstract

Mechanical design is considered a fundamental factor in ensuring the structural integrity and functionality of engineering systems. This research offers a parametric design technique for automating the generation of 3D models of Material Handling Equipment (MHE). By integrating this automated solution into the process, the gap between conceptual design and practical implementation will be bridged. The objective is to streamline the generation process by employing the parametric capabilities of programming-based CAD. A tailored system has been developed to turn natural language user design prompts into precise, modifiable models. The Methodology highlights the deployment of a custom Large Language Model (LLM), trained to generate 3D models with high reusability and scalability, enabling OpenSCAD users of all levels of expertise to have a smooth experience. The work in this research supports the development of cognitive CAD tools, especially where flexibility and customization are vital. The model implemented has been tested on a range of common MHE parts, successfully generating accurate and fully parametric OpenSCAD models. The results demonstrate the model's ability to understand various prompts and produce modifiable outputs suitable for rapid prototyping and design analysis.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the ACLing 2025: 7th International Conference on AI in Computational Linguistics

Keywords: LLM; OpenSCAD; Artificial intelligence and machine learning; Computer-aided design ; Computer graphics;

1. Introduction

The field of computer-aided design (CAD) is undergoing a significant transformation with the rise of programming-based approaches, such as OpenSCAD, which offer better flexibility & parametric control compared direct-manipulation CAD tools [1, 2]. In particular, programming-based CAD tools offer stellar customization capabilities, they also present substantial accessibility challenges for designers who may lack programming expertise. While, programming-based CAD has emerged as a next-generation developing technology in response to the rigid limitations of traditional direct-manipulation CAD editing tools. As this concept is relatively new, the supporting applications,

* Corresponding author. Tel.: +2012-2220-9605

E-mail address: yomna.hassan@guc.edu.eg

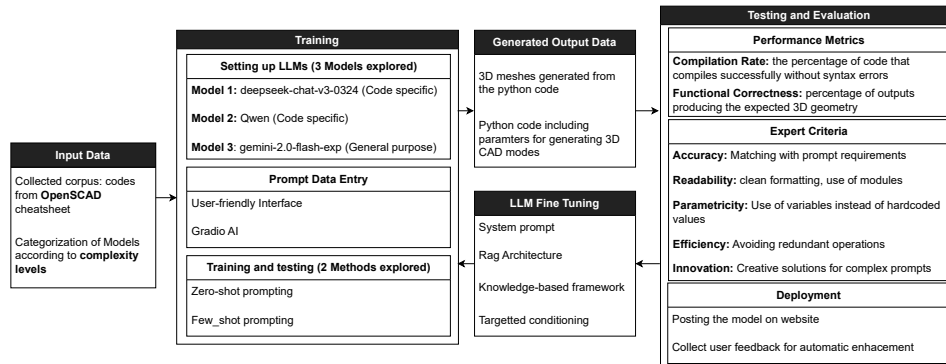


Fig. 1: Block Diagram Describing the Research Work-flow.

such as OpenSCAD, present a steep learning curve [3], particularly for users who primarily engage with CAD design and may find coding challenging. In addition to that, there is only one alternative platform in the field, *zoo.dev*, which provides access to various open-source, community-driven AI models & tools. However, this platform presents major limitations, such as limited knowledge of OpenSCAD libraries and the need for a base code (either provided from the community or by the user) for an accurate generation, which drive amateur users away from using the platform.

The current paper proposes a novel solution that aims to fill the gap between conceptual designs and practical implementation in a seamless way. The framework exploits the development of an intelligent interface that bridges this gap by leveraging large language models to translate natural language descriptions into functional OpenSCAD code, specifically targeting material handling equipment. The main component in the framework involves the development of a Large Language Model (LLM). The model takes input text prompts from the user and outputs the corresponding OpenSCAD code. The LLM is trained on a large set of geometric objects and different types of transformations, allowing the user to utilize the parametric features of OpenSCAD. In the results section, we demonstrate how the framework enabled the users to generate complex and precise models. The use of LLM will aim to eliminate the barriers for non-experts and speed up the design process [4]. On the other hand, the employment of LLMs allowed reaching more precise and customizable models. The main contributions involve: Processing diverse complexities in the context of CAD design, designing an intuitive user interaction scheme with OpenSCAD-based systems to increase accessibility for users, evaluating the effectiveness of text-to-text versus text-to-code model approaches, and reducing LLM hallucinations in CAD model generation.

2. Literature Review

2.1. OpenSCAD Applications and Limitations

Since programming-based CAD tools, such as OpenSCAD, the need to switch from direct-manipulation tools to programming-based tools has emerged. This solves the issue of steep learning curve [3] required to use such tools. This includes drawing simple spheres to drawing shapes from complex mathematical equations. Avila et al. [5] mentioned limitations in rendering, material handling, and a steep learning curve present usability barriers. To address these challenges, further work by Avila et al. [6] explored strategies to improve the user experience during the design process in programming-based CAD applications. A second group of papers examined the application of OpenSCAD in diverse fields, including architecture, additive manufacturing, artifact restoration, tropical geometry, and prosthetics. For instance, OpenSCAD has proven effective in translating mathematical structures into printable 3D models [7]. Its parametric nature allows users to create precise, customizable objects efficiently [1, 2].

Design strategies such as "chunking," which is breaking down complex objects into simpler shapes, have been successful in streamlining OpenSCAD workflows [8]. A notable example is the modeling of Hong Kong's skyline, where individual buildings were deconstructed into basic geometric shapes to capture their architectural essence [9]. To maximize OpenSCAD's parametric capabilities, the use of modular components is emphasized. This allows for minor

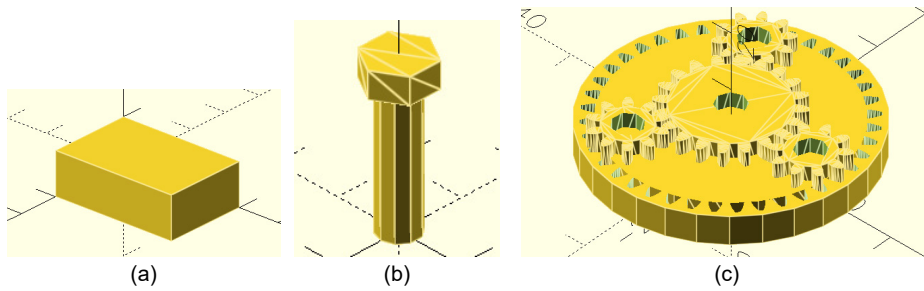


Fig. 2: Examples of models generated from various complexity level prompts. (a)Basic. (b)Intermediate. (c)Advanced.

code modifications post-generation, supporting customization [10, 11]. Despite limitations in flexibility and complexity, OpenSCAD's primitive construction and transformation functions have proven effective for detailed shaping and structural design [12]. Finally, inverse design strategies using machine learning have been introduced to optimize model parameters and geometry for target performance objectives [4]. This further underscores the growing synergy between AI technologies and parametric CAD modeling.

2.2. AI-Assisted Code Generation

One group of studies focused on AI-powered code generation for various applications and languages. These works investigated the use of Large Language Models (LLMs), such as GPT-4, in translating natural language prompts into design requirements, proposing ideas based on performance criteria, and supporting generative design workflows [13].

While LLMs and Vision-Language Models (VLMs) were used to generate CAD code, concerns were raised regarding their reliability. A recurring question was: "Why not use ChatGPT directly for simplicity?" The primary issue lies in a phenomenon known as *AI hallucination*, where an AI fills in missing knowledge with plausible but incorrect information [14]. This can result in syntactically incorrect or structurally flawed outputs. To mitigate this, the concept of *prompt engineering* was introduced. By crafting prompts with increasing specificity, researchers were able to reduce ambiguity and improve output quality [15]. Further improvements were observed using the *chain-of-thought* technique, decomposing complex problems into smaller tasks and solving them iteratively [16, 17, 18]. Tokenization techniques were shown to enhance the model's ability to process complex text sequences [19]. For example, a CAD-generating application developed by Zoo utilized a web API and scripting-based user prompts to produce modifiable solid-body models [20]. These models are not decomposed into sketches and extrusions but rather rendered as complete bodies. An innovative framework using VLMs for CAD verification and refinement was introduced by AI Rashedy et al. [21]. This approach continuously evaluates the generated 3D structures, enhancing model integrity and reliability. Through sequential text-and-image prompting, AI systems encoded architectural elements into OpenSCAD-compatible scripts, demonstrating strong potential for automated design workflows [22].

3. Methods

3.1. Dataset Preparation

As shown in Fig. 1, the dataset was aggregated from the most used OpenSCAD functions collected from the official OpenSCAD website [23]. The data set consists of functions with varying complexity levels (simple, intermediate, and advanced), which are determined based on the number of components entailed in these functions.

3.2. LLM Implementation Process

The LLM setup is unified across all three models to ensure that the evaluation process later on will be as fair as it could. A hybrid setup is used to reach an optimal development and testing experience. This setup has been put into

place using VSCode & Google Colab. This approach utilizes the qualities of both environments to serve the needs of the project. VSCode provides a more efficient development process through the robust IDE features, and the GPU access is dependent on the local machine's GPU, unlike colab where the GPU is limited to the subscription tier. In addition to that colab has no persistent storage features. However, colab offers a strong testing environment, since the testing process does not require prior setup or any dependencies and provides a smooth experience. A virtual environment, **.venv**, ensures dependency isolation, which is critical for ML projects, to ensure that the project is runnable on colab smoothly after migration. A critical feature **.venv** offers, is the automatic generation of a `requirements.txt` file, which is then used in colab to streamline the process of setting up all the required dependencies for the project to run. Another crucial feature **.venv** offers, is that it eliminates the need for manual source commands, meaning it activates the environment automatically upon running the project. The integration of a virtual environment into this project will bridge the gap between local development and cloud execution.

3.2.1. LLM Setup

The system integrates the OpenAI client with OpenRouter by configuring a customized base URL and timeout, while securely loading API keys using the `dotenv` library. To efficiently handle large OpenSCAD code outputs, chunked streaming is implemented, allowing partial responses to be concatenated in real time, which accelerates response delivery and prevents timeouts. The prompt processing phase includes validating that user input is a non-empty English string. Then, messages are sent to the LLM API with an exponential back-off retry mechanism—allowing up to three attempts with doubling wait times between retries to handle transient failures. Extracted OpenSCAD code blocks are handled by scripts that recognize preferred, alternative, and fallback code patterns to accommodate different model response formats. This ensures graceful fallback in rare cases of uncommon formats. Post-generation, code is saved as a timestamped `.scad` file in an `Outputs` folder with error handling for any saving issues. Logging is used extensively at multiple levels: `logging.debug` captures full prompts for debugging during development, whereas `logging.exception` automatically logs errors and stack traces to facilitate debugging and maintenance. This setup ensures robustness, efficiency, and traceability in the code generation workflow.

3.3. Model Selection and Training

Several models are compared as follows. The Open Router platform offers multiple LLM choices, but some models like `mistral-small-3.1-24b-instruct:free` show hallucinations by fabricating functions or misinterpreting syntax in OpenSCAD code generation. The `meta-llama/llama-3.3-70b-instruct:free` model, while strong in reasoning, produces overly generalized responses and thus lacks reliability for precise OpenSCAD code generation. Based on initial experiments, three LLM models were selected for evaluation and integration based on their performance: `google/gemini-2.0-flash-exp:free`, `deepseek/deepseek-chat-v3-0324:free`, and `qwen/qwq-32b`. The first is for general-purpose code, while the other two are for code-specific ones.

Zero-shot prompting is used to allow the model generate responses relying solely on pre-trained knowledge without training data. This provides a baseline to identify model weaknesses. Few-shot prompting is used to allow the model to receive a few input-output examples as `System Prompt` to teach OpenSCAD syntax and parametric constraints. This enhances the performance without full model training.

DeepSeek chat v3 was chosen for integration into the final workflow due to its highest performance among tested models. Prompt engineering training is applied to make up for the few-shot training suffices in case of large-scale production.

3.4. LLM Fine-tuning

Key challenges identified include handling complex designs with multiple interconnected modules. The model also struggled with performing 3D operations on 2D shapes. Additionally, overall code generation exhibited notable inconsistency. To address these limitations, more datasets addressing identified model limitations were collected from open-source libraries, mostly as ready-made JSON files, with some preprocessed OpenSCAD files converted to JSON for training purposes. The training approach combines `System Prompt` and RAG architecture methods to optimize results. The paragraphs below showcase the various types of fine-tuning conducted.

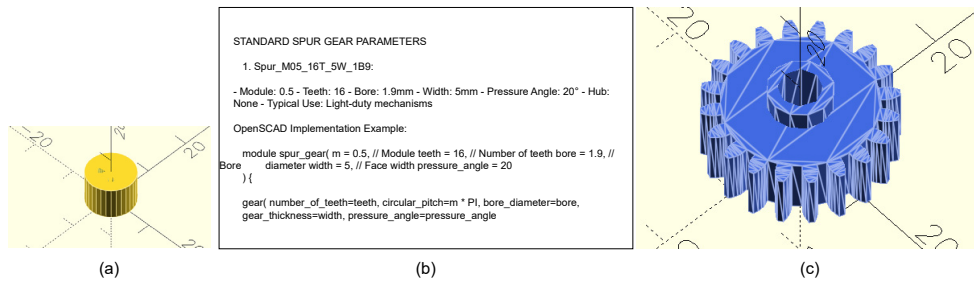


Fig. 3: The effect of "system prompt" on the generated model (a) Model resulted from training without "system prompt" (b) Code generated after adding "system prompt". (c) The model generated after incorporating "system prompt".

System Prompt: It provides clear instructions to the LLM on its role as an expert OpenSCAD assistant, operational guidelines, and strategies to mitigate known limitations. It guides the model to use the RAG architecture prior to processing design prompts, ensuring consistent and context-aware responses.

RAG Architecture: A knowledge_base was built from `.txt` and `.json` files containing core rules, guidelines, preprocessed code, history, and error handling. Texts were chunked and encoded into **vector_embeddings** using SentenceTransformer and stored in the knowledge_base. Upon design prompt input, an embedding of the prompt is compared to the stored embeddings to find relevant chunks, which augment the prompt before LLM processing.

Knowledge-Base Framework: The framework is composed of several modules to enforce primary standards, such as: modern syntax, variable declaration before use, manifold geometry, MCAD standards, naming conventions, parametric flexibility, and guiding comments to resolve issues like misalignment and misuse of 3D operations.

Targeted Conditioning: It is implemented for model-specific enhancement. It involves complex designs like gears, referencing gear guidelines and datasets (e.g., **Gears.JSON** from thingiverse), including advanced gears datasets.

Adaptive Learning: User feedback on generated code correctness is collected to update a **History.JSON** file, which the model consults before applying general guidelines for improved consistency.

3.5. Data Entry User Interface

Gradio was chosen as the front-end framework to develop a user-friendly interface for the final workflow due to its seamless integration with Python. It also provides extensive support for machine learning and large language model applications. Besides, it offers a comprehensive set of features such as real-time streaming, persistent chat history, multi-modal outputs, and interactive components. Gradio also provides flexible deployment options, including public sharing links without server setup, local hosting, Hugging Face spaces, and Docker containers for production. Several features were implemented in the Gradio app. **Design Entry & Code** is allowed through a user input area for design prompts with an example, plus a generated code display area to preview code before rendering. **File Operations** enable automatic creation of an **Outputs** folder if missing, automatic download of `.scad` file inside it, and launching OpenSCAD locally for user verification. Additional buttons provided to open **Outputs** folder, launch OpenSCAD, and download files in case of failure. **Feedback** is collected after code generation. For this purpose, a feedback form appears for users to evaluate the response; feedback decides whether the response is saved to **History.JSON** based on correctness.

4. Experiments and Evaluations

4.1. Evaluation Framework

A combination of tests covering multiple aspects is used to evaluate the performance of each model. In order to standardize the test suite, a set of 30 test cases has been developed with each case designed to target one or more of the functional dimensions previously outlined. Prompt complexity levels are used where prompts are categorized into 3 levels: Basic prompts (Example: Generate a cube with length 20mm, width 30mm, and height 10mm , Intermediate

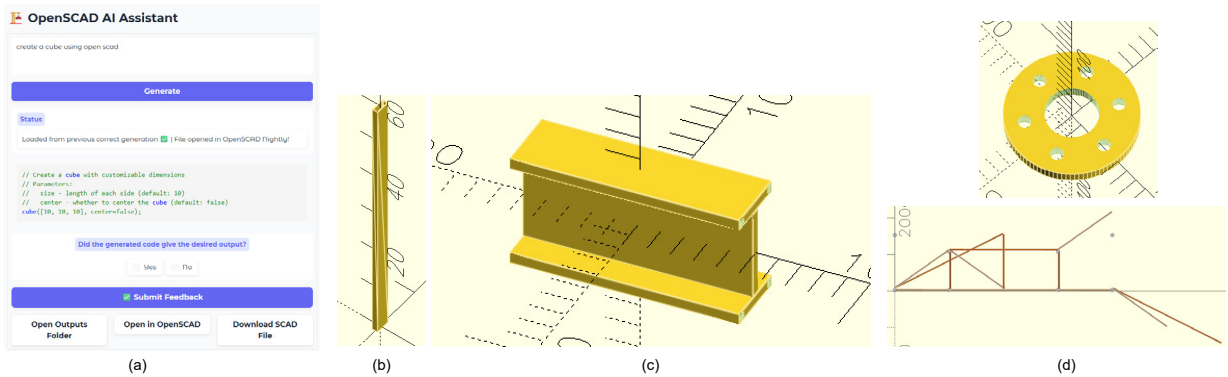


Fig. 4: The figure demonstrates several results. (a) demonstrates a UI with prompt used for the generating a python code that produces a beam model (b) A model generated by a python code without integrating a fall back prompt (c) Model Generated after applying the fall back prompt layer. (d) Issues handled according to expert evaluation. The upper figure represents a sample of Flange handled correctly outside of training scope. The lower figure represents a sample of Trusses that are handled incorrectly.

Prompts (Example: Create an M6 bolt with 20mm length and hexagonal head), and advanced (Example: Design a planetary gear system with 3 planet gears). For each of these categories, 10 test cases are used to assess each model, ensuring consistency and standardization across the evaluations. This evaluation framework assesses the syntax validity, functional correctness, and parametric flexibility of OpenSCAD code generated by different models to benchmark their overall performance. Performance metrics are demonstrated in Fig. 1.

4.2. Expert Evaluation setup and criteria

OpenSCAD subject matter experts (SMEs) compared the research with the cloud-based CAD modeling tool *zoo.dev*, which supports AI-generated design pipelines and enables design creation/editing via natural language in the browser. While *zoo.dev* offers features like model comparison and geometry inspection, it lacks support for *.scad* files and only outputs images, text, or standard data types. This limits low-level debugging and real-time customization. Additionally, *zoo.dev* requires a reliable starting code, posing a challenge for amateur engineers. It also does not support OpenSCAD-specific syntax or parametric designs. The experts tested the system with their own cases. They evaluated the model on a 1-to-5 scale across five criteria as shown in Fig. 1. A *colab*-based testing environment was provided to facilitate evaluation, but experts encountered difficulties deploying the application correctly. To address this, a workaround was implemented by using *ngork*. *Ngork* enables remote internet access to locally running apps and APIs without requiring port forwarding or server deployment by establishing a secure, encrypted tunnel from *localhost* to the internet. This approach proved reliable for rapid prototyping.

5. Results and Discussion

Upon conducting the previously defined tests for quantitative results on all three models, the results assisted in deciding which model is best suited to be used in the final workflow. Table 1 showcases the summary. *Gemini pro-exp* showed promising performance that required mild modifications for the initial 10 test cases corresponding to the basic prompt complexity level. As for the intermediate prompts, the generated codes were mostly syntactically correct, and parametric flexibility was satisfied with all the designs, yet barely any functional correctness was shown. However, the model failed to generate consistently reliable or functional codes for the advanced prompt. In contrast, *Qwen 32B* delivered relatively stronger performance with basic prompts, where generated codes were syntactically and functionally correct. For intermediate prompts, although the codes produced were generally syntactically correct and executable, they did not consistently render the intended designs. At the advanced complexity level, the model was unable to produce functionally usable code. Although the model was not able to consistently deliver well-structured functional codes, the model showed strong reasoning capabilities with design breakdowns. The third model *DeepSeek chat v3*, delivered the most consistent performance across all levels of prompt complexity. This model has successfully generated valid and functionally accurate code for most of the basic, intermediate, and advanced test cases. While

Table 1: Comparative Analysis of Models Across Test Complexities.

Test	Gemini pro-exp			Qwen 32B			DeepSeek chat v3		
	Basic	Intermediate	Advanced	Basic	Intermediate	Advanced	Basic	Intermediate	Advanced
Compilation Rate	80%	90%	0%	100%	90%	0%	100%	90%	80%
Functional Correctness	70%	40%	0%	90%	40%	0%	100%	90%	60%
Parametric Correctness	100%	60%	0%	100%	60%	0%	100%	100%	100%

some outputs showed that the model has some limitations that require further training, the overall performance indicates that this is the most suitable candidate for integration into the final workflow, with only mild prompt engineering needed to optimize its outputs.

5.1. Qualitative Observations

Based on the results mentioned above, DeepSeek chat v3 is the most suitable candidate to be integrated into the main workflow. Some key limitations were observed in the model, and accordingly System_prompt was developed.

Error Patterns: Despite overall good performance, DeepSeek chat v3 exhibited persistent errors with complex designs, including imprecise spatial translations causing misalignment in multi-module designs, misuse of 3D operations on 2D shapes due to improper OpenSCAD library usage, difficulty generating correct parameters for complex designs like gears (leading to incomplete or non-functional models), and using reserved words as variables, which caused compilation errors and inconsistent outputs.

Impact of Prompt Engineering: Incorporating the System Prompt into the Python script for LLM deployment resulted in clear improvements in output quality and consistency. The LLM's responses improved when reassessed by previous criteria, resolving issues such as misalignment and misuse of 3D operations. Additionally, gear generation accuracy improved after implementing a gear-targeted conditioning mechanism within the System Prompt. These improvements in misalignment and gear handling are demonstrated in figures.

5.2. Expert Evaluation Results

Five OpenSCAD experts evaluated the interface using their prompts and rated the system across five dimensions from 1 (poor) to 5 (excellent). The model achieved a total score of 20 out of 25 (80% success rate), exceeding the 75% target. Positive feedback was received for models within the training scope (e.g., gears), where generated code was accurate and parametric. For models outside the training scope (e.g. flanges, Fig. 4 (c)), the model provided good calculations with varying accuracy, saving engineers time by providing baselines. The model successfully generated missing parameters and performed calculations when user prompts lacked them, addressing a common issue for OpenSCAD users struggling with parametric mathematical expressions [24]. The app (Fig. 4 (a)) facilitates users of different expertise levels to develop OpenSCAD designs smoothly by providing baseline designs, contrasting with zoo.dev, which failed to generate reliable code even with similar prompts. The system effectively reuses code from prior requests, leveraging RAG integration and history files, providing an advantage over zoo.dev, which suffers from cloud-sourcing issues. The generated code is logical, well commented, readable, maintainable, and usable for real-world engineering. On the limitations side, the model consistently generated incorrect I-beams (normal beams) despite repeated attempts (Fig. 4). Complex designs like queen post trusses had correct module code but flawed positioning and alignment (Due to applying incorrect operations), requiring significant edits (Figure 4 (c)). The generated complex designs were detailed and modular, aiding maintenance despite needed corrections. The experts noted that the model generated rigid responses for complex design prompts outside its training scope. This was due the lack of fallback when RAG retrieved no relevant files. To handle this issue, a detailed fallback system prompt was added for those cases. RAG retrieval rules are now optimized to append only relevant files to user prompts, replacing the previous generalized approach. After applying the modifications mentioned, an improvement was noticed, specifically for models that are out of the training scope as shown in Fig. 4 (b) and (c).

6. Conclusions and Future Work

In this work, a new framework was proposed that provides a seamless user experience for CAD designers with limited programming skills. A mix of the parametric capabilities of OpenSCAD and the potential of LLMs in automating programming was utilized. Several LLM models were compared to decide the best approach. The results demonstrated how beginners could generate complex models quickly and easily using the proposed framework. As future work, cloud hosting can be used for the `knowledge_base` instead of local storage, for faster retrieval of relevant documents. Also, the use of multi-turn conversations could be added to help users edit code automatically.

Acknowledgements

We would like to extend our thanks to Adhoc Designs (Cairo, Egypt) for assisting in the testing phase of the project.

References

- [1] C. Wang, Chinese pastry design and implementation based on 3d modeling, *Lecture Notes in Education Psychology and Public Media* (2023).
- [2] P. Salvi, B. Németh, *Modeling acorns* (2024).
- [3] L. Zheng, Restore traditional chinese lanterns based on openscad and threejs, in: *2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023)*, Atlantis Press, 2024, pp. 526–537.
- [4] L. Makatura, M. Foshey, B. Wang, F. Hahnlein, P. Ma, B. Deng, W. Matusik, How can large language models help humans in design and manufacturing? part 1: Elements of the llm-enabled computational design and manufacturing pipeline, *Harvard Data Science Review* Special Issue 5 (2024).
- [5] J. Gonzalez Avila, T. Pietrzak, A. Girouard, G. Casiez, Understanding the challenges of openscad users for 3d printing, in: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–20.
- [6] J. G. Avila, *Facilitating programming-based 3d computer-aided design using bidirectional programming*, Ph.D. thesis, Université de lille; Carleton University (CA) (2024).
- [7] H. Gangl, Y. Ren, Z. Urbancic, *Hands-on Tropical Geometry*, "arXiv", 2023.
- [8] D. Yuan, A cross-software computer graphics design of chinese biscuit based on openscad and shaderfrog, *Communications in Humanities Research* (2023).
- [9] Z. Zhang, Research of modeling skyline of hongkong central with openscad and blender, *Applied and Computational Engineering* 110 (2024) 55–64.
- [10] A. Fini, C. Brooks, A. Romani, A. Straatman, J. Pearce, Parametric design of easy-connect pipe fitting components using open-source cad and fabrication using 3d printing, *Journal of Manufacturing and Materials Processing* 9 (2) (2025) 65.
- [11] F. Răduică, I. Simion, The design of an upper-limb prosthetic finger using algorithms for full customization, no publication details provided.
- [12] Z. Guo, Research and design of chinese dragon based on openscad, *Transactions on Computer Science and Intelligent Systems Research* (2024).
- [13] L. Makatura, M. Foshey, B. Wang, F. Hahnlein, P. Ma, B. Deng, W. Matusik, How can large language models help humans in design and manufacturing? part 2: Synthesizing an end-to-end llm-enabled design and manufacturing workflow, "" (2024).
- [14] M. Nejjar, L. Zacharias, F. Stiehle, I. Weber, Llms for science: Usage for code generation and data analysis, *Journal of Software: Evolution and Process* 37 (1) (2025) e2723.
- [15] L. Murr, M. Grainger, D. Gao, Testing llms on code generation with varying levels of prompt specificity, *arXiv preprint arXiv:2311.07599* (2023).
- [16] H. Deng, S. Khan, J. Erkoyuncu, An investigation on utilizing large language model for industrial computer-aided design automation, *Procedia CIRP* 128 (2024) 221–226.
- [17] B. Jones, F. Hahnlein, Z. Zhang, M. Ahmad, V. Kim, A. Schulz, A solver-aided hierarchical language for llm-driven cad design, *arXiv preprint arXiv:2502.09819* (2025).
- [18] N. Surovi, P. Witherell, Generative artificial intelligence (genai) prompt engineering for additive manufacturing (am) (2024).
- [19] M. Raiaan, M. Mukta, K. Fatema, N. Fahad, S. Sakib, M. Mim, S. Azam, A review on large language models: Architectures, applications, taxonomies, open issues and challenges, *IEEE Access* 12 (2024) 26839–26874.
- [20] M. Khan, S. Sinha, T. Sheikh, D. Stricker, S. Ali, M. Afzal, Text2cad: Generating sequential cad models from beginner-to-expert level text prompts, *arXiv preprint arXiv:2409.17106* (2024).
- [21] K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu, M. Gombolay, Generating cad code with vision-language models for 3d designs, *arXiv preprint arXiv:2410.05340* (2024).
- [22] J. Huang, A. Haridis, Evaluation of architectural synthesis using generative ai, *arXiv preprint arXiv:2503.02861* (2025).
- [23] OpenSCAD, *Openscad cheatsheet*, <https://openscad.org/cheatsheet/> (2025). URL <https://openscad.org/cheatsheet/>
- [24] J. F. Gonzalez, T. Pietrzak, A. Girouard, G. Casiez, Facilitating the parametric definition of geometric properties in programming-based cad, in: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ACM, Association for Computing Machinery, New York, NY, USA, 2023. doi:10.1145/3544548.3580682.